BuMoChi User Guide

BuMoChi

BuMoChi User Guide

This is a user guide draft for the current Bmc interface. It outlines the purpose of the BuMoChi library, the applications it works with, and gives examples of commands for doing basic tasks with this library like recording, playback and composition of animation clips.

For installation and getting started see files [[Documentation/BuMoChi/Installation|Installation]] and [[Getting Started]].

1. What BuMoChi is

BuMoChi is a SuperCollider library for performing with motion-capture data and animated characters. It lets a live coder treat movement as material: receive it from a camera, record it, replay it, change its timing, combine selected body parts, and send the result to an animated character. BuMoChi also supports live collaboration among separate performance workstations, each with its own motion-capture tracking, computer, sound-generation, and animation systems. By sharing motion-capture data as OSC through OSCGroups, it makes it possible to rehearse and perform together from different locations over the network.

The library is intended for networked dance, theatre, music, animation, and other performance situations in which movement should remain open to live intervention. A performer can be captured in one place while collaborators process the movement or render the avatar somewhere else.

The main user interface is the `Bmc` class. Common actions are deliberately short enough for live coding:

```
Bmc.record(\entrance);
Bmc.stopRecording;
Bmc.play(\entrance);
Bmc.rate(0.5);
Bmc.loop(true);
```

You do not need to understand the internal classes before using these commands. The supporting classes described later in this guide are available when you need more detailed control.

What can an end user do?

With the current Bmc interface you can:

- receive live skeletal motion from XR Animator;
- route motion by source and avatar name;
- display the motion on a VMC-compatible Godot character;
- record motion as named clips;
- list, select, rename, save, load, and remove clips;
- play, pause, seek, loop, and change the speed of clips;
- copy selected bones or body regions between recordings;
- build a live composite avatar from different motion sources;
- inspect whether frames are arriving or being rejected.

BuMoChi operates inside SuperCollider, but using the movement features does not require booting SuperCollider's audio server. The audio server becomes relevant when a performance also synthesizes sound or uses synth control buses to change movement.

2. The software connected by BuMoChi

The complete networked pipeline is:

```
XR Animator
    → BunrakuOSCEncoder.py
    → local SuperCollider + BuMoChi and OSCGroups
OSCGroups
    → every collaborator's SuperCollider + BuMoChi
each local BuMoChi
    → local BunrakuOSCDecoder.py
    → local Godot
```

On a single computer, OSCGroups can initially be omitted:

```
XR Animator → encoder → BuMoChi → decoder → Godot
```

The components have different jobs. XR Animator observes a person, and the encoder packages one complete route-free skeleton frame. It sends identical copies to local BuMoChi and to OSCGroups. OSCGroups distributes these source frames to the other workstations, where they also enter BuMoChi. Each local BuMoChi records, combines, transforms, and assigns all available sources to figures and avatars, then sends its completed scene only to its own decoder and Godot renderer.

Distributed sources, local synthesis

BuMoChi uses a distributed-sources, local-synthesis model. What travels between collaborators is motion source material, not a finished rendered animation. Every workstation receives its own motion source directly and remote motion sources through OSCGroups. Its local SuperCollider/BuMoChi process then creates the complete animation scene from those inputs, using the same session definition as the other workstations. Finally, it routes the completed avatars through its local decoder to its local Godot scene.

This separation is fundamental. OSCGroups is the shared source-data layer; BuMoChi is the local animation synthesis layer; Godot is the local renderer. Bmc's processed routed frames never return to OSCGroups. When collaborators load the same session and Godot scene, they synthesize and render the same defined performance independently, in the same way that sc-hacks-redux shared control sources while each workstation synthesized sound locally.

SuperCollider and BuMoChi

SuperCollider is the live-coding environment in which BuMoChi runs. Download SuperCollider from its official site:

- https://supercollider.github.io/downloads

The BuMoChi repository contains the library itself. Place or link the BuMoChi repository folder inside your SuperCollider user extensions directory. To see that directory, evaluate:

```
Platform.userExtensionDir;
```

After installing or updating BuMoChi, recompile the SuperCollider class library. In the SuperCollider IDE, use **Language → Recompile Class Library**.

The current source files for the public interface are under:

```
Classes/Bmc/
```

XR Animator

XR Animator uses a webcam to track a performer and animate a humanoid model. In this pipeline it is the source of standard VMC motion messages.

- Project and documentation: https://github.com/ButzYung/SystemAnimatorOnline
- Native application releases: https://github.com/ButzYung/SystemAnimatorOnline/releases
- Online version: https://sao.animetheme.com/XR_Animator.html

Use the native Electron application for this pipeline because VMC output is a native-application feature. Configure its VMC destination host and port to match the encoder; the local hello-world example below uses `127.0.0.1:39537`.

Python encoder and decoder

The two Python command-line scripts described here can be started together with the one-command shell launcher described in Setup.

The required Python command-line scripts are included in the BuMoChi distribution:

```
PipelineApplications/BunrakuOSCEncoder.py
PipelineApplications/BunrakuOSCDecoder.py
```

Their supporting Python modules are in the same directory. Keep those files together. The scripts use Python 3 and do not require third-party Python packages.

BunrakuOSCEncoder receives VMC from XR Animator. It collects the 21 required humanoid bones and sends each route-free `/bunraku/vmc/frame` both to local Bmc on `57130` and to local `OscGroupClient` on `22244`. Remote clients deliver their route-free frames to Bmc on `57130`. Bmc synthesizes the complete local scene, adds final avatar routes, and sends routed output only to the local decoder on `39538`.

BunrakuOSCDecoder performs the reverse conversion. It receives Bunraku frames from SuperCollider and emits standard VMC messages for Godot.

An additional, more general VMC packet-preserving bridge is included under:

```
HelperAppsAndExamples/VMC_Converter_Scripts/
```

The Bmc classes documented here currently use the fixed Bunraku Frame protocol-v1 encoder and decoder in `PipelineApplications`.

OSCGroups

OSCGroups carries OSC messages between collaborators. All participants connect to the same OSCGroups server and group. Each participant uses a local send port and a local receive port; applications continue sending ordinary UDP OSC to localhost.

BuMoChi includes OSCGroups documentation and some prebuilt clients under:

```
HelperAppsAndExamples/OSCGroups/
```

Upstream source and build instructions are available from:

- https://github.com/RossBencina/oscgroups
- https://github.com/RossBencina/oscpack
- http://www.rossbencina.com/code/oscgroups

You do not need OSCGroups for the first local test. Introduce it after the encoder-to-BuMoChi-to-decoder path works on one computer. Detailed staged tests are in `PipelineApplications/Communication_Tests/`.

The *`OscGroupClient`* application can optionally be started by the same one-command shell launcher described in Setup.

Godot

Godot renders the animated character. Download a current compatible Godot 4 editor from the official site:

- https://godotengine.org/download/

A VMC-compatible reference project is included at:

```
PipelineApplications/GodotVMCReference/project.godot
```

Import that file from the Godot Project Manager and run the project. The reference test configuration listens for reconstructed VMC on UDP port `39539`.

You may later replace the reference character or project. As long as the new Godot project accepts standard VMC and has a correctly mapped humanoid skeleton, the encoder, BuMoChi, and decoder do not need to change.

3. Hello world: capture, record, and replay movement

This first example runs everything on one computer and deliberately omits OSCGroups. It demonstrates the smallest useful BuMoChi session.

Port map

From	To	UDP port
XR Animator	Python encoder	39537
Python encoder	BuMoChi	57130
BuMoChi	Python decoder	39538
Python decoder	Godot	39539

Only one program can listen on a given UDP port. Stop older test processes before starting this example.

Step 1: start Godot

Import and run:

`PipelineApplications/GodotVMCReference/project.godot`

Step 2: start the decoder

Open a terminal in `PipelineApplications` and run:

```
python3 BunrakuOSCDecoder.py \
  --listen-port 39538 \
  --accept-avatar "BunrakuTestAvatar" \
  --verbose
```

Step 3: prepare BuMoChi in SuperCollider

Evaluate this block:

```
(
Bmc.reset;

// This name must match the encoder's --avatar value.
Bmc.addAvatar(\BunrakuTestAvatar, "BunrakuTestAvatar");
Bmc.selectAvatar(\BunrakuTestAvatar);

// Route this avatar through the local decoder to Godot.
Bmc.avatar(\BunrakuTestAvatar).vmcPort_(39539);

// Receive Bunraku frames from the Python encoder.
Bmc.start(57130);
)
```

Check the state:

```
Bmc.status;
```

The posted event should include `running: true` and `port: 57130`.

Step 4: start the encoder

See Setup for the easiest way to start the encoder.

Manual procedure:

In a second terminal, also opened in `PipelineApplications`, run:

```
python3 BunrakuOSCEncoder.py \
  --no-oscgroups \
  --avatar "BunrakuTestAvatar" \
  --source "xr-animator" \
  --verbose
```

Step 5: start XR Animator

In XR Animator, set the VMC destination to:

```
Host: 127.0.0.1
Port: 39537
```

Enable VMC output and move in front of the camera. The Godot character should follow the motion. Evaluate `Bmc.status` again; the `received` count should be increasing.

Step 6: record a clip

Start recording in SuperCollider:

```
Bmc.record(\hello, "BunrakuTestAvatar", "xr-animator");
```

Move for several seconds and then stop:

```
~helloClip = Bmc.stopRecording;
```

Inspect the result:

```
~helloClip.size;
~helloClip.duration;
Bmc.listClips;
```

Step 7: replay the clip

Disable XR Animator output, or simply stand still, and evaluate:

```
Bmc.play(\hello);
```

Try changing playback:

```
Bmc.rate(0.5);     // half speed for the next playback
Bmc.loop(true);
Bmc.play(\hello);

Bmc.pause;
Bmc.resume;
Bmc.stopPlayback;
```

Step 8: shut down

```
Bmc.stop;
```

Then disable XR Animator output, press **Control-C** in the encoder and decoder terminals, and stop the Godot project. `Bmc.stop` stops the BuMoChi receiver and playback; it does not stop the external applications.

4. Bmc method reference

`Bmc` is the recommended entry point for ordinary sessions. The methods below delegate work to the appropriate supporting object.

System control

After the SuperCollider class library compiles, Bmc automatically listens for route-free Bunraku frames on input port `57130`. Its selected default avatar is `Ishidomaru`; completed frames for that identity are rewritten as `Ishidomaru`, assigned Godot VMC destination port `39539`, and forwarded to the local Bunraku decoder on port `39538`. Decoder forwarding is enabled. This ready state supports immediate live monitoring and `Bmc.record` without a separate initialization block when the encoder uses `--avatar "Ishidomaru"` and Godot listens on `39539`.

1. `Bmc.start(port: 57130)`

   Starts or restarts the Bunraku Frame OSC receiver. The receiver starts automatically on `57130` after class-library compilation; call this method to restart it after `Bmc.stop` or to select another input port. The port must match both the local encoder output and `OscGroupClient.localRxPort`.

2. `Bmc.stop`

   Stops clip playback, cancels an unfinished recording, and closes the BuMoChi OSC receiver. It does not stop XR Animator, Python, OSCGroups, or Godot.

3. `Bmc.status`

   Posts and returns an event containing receiver, recording, playback, clip, and wire statistics. Useful keys include `running`, `port`, `received`, `rejected`, `dropped`, `recording`, `playing`, `currentClip`, and `clipCount`.

4. `Bmc.help`

   Posts and returns a compact port-configuration summary for XR-Animator, `BunrakuOSCEncoder`, SuperCollider/Bmc, `BunrakuOSCDecoder`, and every avatar that currently has a Godot VMC output port.

   ```
   Bmc.help;
   ```

   The paired encoder/SuperCollider input lines reflect the dispatcher's current port. The paired SuperCollider output/decoder input lines reflect `Bmc.decoderPort`. Avatar output pairs reflect the current `BmcAvatar.vmcPort` settings, so the text updates after runtime configuration changes.

5. `Bmc.showDispatcherStatus(updateInterval: 0.25)`

   Opens the OSC/VMC input monitor window. A static field identifies the monitored OSC address and the dispatcher's configured UDP port. A dynamic field displays the latest `BmcDispatcher.status` dictionary and refreshes every `0.25` seconds by default. Supply another interval in seconds to change the refresh rate. Closing the window stops its update routine.

   ```
   Bmc.showDispatcherStatus;      // refresh four times per second
   Bmc.showDispatcherStatus(1.0); // refresh once per second
   ```

6. `Bmc.reset`

   Stops the current session, replaces the working clip library, avatars, recorder, player, dispatcher, and wires with fresh objects, restores the Ishidomaru/`39539` default route, and restarts the receiver on `57130`. Unsaved in-memory clips are lost, so use it deliberately.

Avatars and output

1. `Bmc.addAvatar(name, displayName)`

   Creates an avatar destination. Its name should match the avatar name carried by incoming frames when it is intended to receive that stream directly.

2. `Bmc.avatar(name)`

   Returns a registered `BmcAvatar`. With no argument it returns the default avatar, `Ishidomaru`.

3. `Bmc.selectAvatar(name)`

   Makes an avatar the destination for subsequent playback and completed-frame recording.

4. `Bmc.output(destination)`

   Replaces the selected avatar's default local-decoder output function with a custom destination. This is an advanced escape hatch.

   ```
   Bmc.output(NetAddr("127.0.0.1", 39538));
   ```

   A function may also be used as an output for inspection or custom processing.

5. `Bmc.decoderPort_(port)`

   Sets the local decoder destination for all avatars using the default output. Default: `39538`.

6. `Bmc.forwardDecoder_(flag)`

   Enables or disables forwarding to the local decoder. Default: `true`.

   These are Bmc's only class-wide network-output controls. Source distribution to OSCGroups belongs to BunrakuOSCEncoder, not Bmc.

7. `Bmc.sendCalibrationFrame(port)`

   Sends one approximate upright T-pose for the selected avatar to a local decoder input. If `port` is omitted, Bmc uses its currently configured decoder port, which defaults to `39538`. The routed frame separately embeds the selected avatar's Godot VMC destination, which defaults to `39539` for Ishidomaru.

   ```
   Bmc.sendCalibrationFrame;       // decoder input 39538 by default
   Bmc.sendCalibrationFrame(39548); // explicitly use another decoder input
   ```

   This is a pipeline-connectivity test, not a calibrated reference pose for a particular VRM model.

Recording

1. `Bmc.record(name, avatar, source, capturePoint, metadata)`

   Begins a recording. Every argument is optional. SCD is the default recording format: when `Bmc.stopRecording` is called, the completed clip is retained in memory and automatically saved as `name.scd` in `BmcClipLibrary.defaultDirectory`.

   ```
   Bmc.record;                          // record all incoming frames
   Bmc.record(\take1);                  // record all frames as \take1
   Bmc.record(\take1, "actor", "camA");
   ```

   `capturePoint` is `\rawFrame` by default. Use `\completedFrame` to record the selected avatar after reference-pose completion and live wiring.

2. `Bmc.recordScd(name, avatar, source, capturePoint, metadata)`

   Explicit alias for the default `Bmc.record` behavior.

3. `Bmc.recordBmc(name, avatar, source, capturePoint, metadata)`

   Begins a recording that will be saved in the legacy `.bmc` archive format when stopped. Use this only when that format is specifically required.

4. `Bmc.stopRecording`

   Stops recording, returns a `BmcMocapClip`, adds it to the in-memory clip library, makes it the current clip, and saves it to disk in the format selected when recording began. Ordinary `Bmc.record` and `Bmc.recordScd` calls save `.scd`; `Bmc.recordBmc` saves `.bmc`.

5. `Bmc.cancelRecording`

   Stops recording and discards the frames collected in that take.

6. `Bmc.isRecording`

   Returns `true` or `false`.

Clip library

1. `Bmc.clips`

   Returns the dictionary of all in-memory named clips.

2. `Bmc.clip(name)`

   Returns a named clip. With `nil`, it returns the current clip.

3. `Bmc.selectClip(name)`

   Makes a named clip current and returns it.

4. `Bmc.currentClip`

   Returns the currently selected clip.

5. `Bmc.listClips`

   Posts clip names, frame counts, and durations in the SuperCollider post window. The current clip is marked with `*`.

6. `Bmc.showClips`

   Opens the clip window. Initially it shows clips currently loaded in memory. The buttons above the list provide two disk and playback operations:

   - `List saved` scans `BmcClipLibrary.defaultDirectory` for `.scd` and `.bmc` files and displays their names without loading their contents into memory.
   - `Play selected` loads the selected saved clip if necessary, then begins playback. A clip already in memory is played directly.

   Selecting a row for an in-memory clip also makes it the current clip. Saved clips shown by `List saved` remain unloaded until `Play selected` is pressed.

7. `Bmc.renameClip(oldName, newName)`

   Renames a clip in the in-memory library.

8. `Bmc.removeClip(name)`

   Removes a clip from memory. This does not delete a separately saved file.

9. `Bmc.saveClip(name, path)` / `Bmc.save(name, path)`

   Writes a clip archive. If the path is omitted, BuMoChi uses a `BmcClips` directory inside `Platform.userAppSupportDir` and the `.bmc` extension.

10. `Bmc.loadClip(path, name)` / `Bmc.load(path, name)`

    Loads a saved `.bmc` or `.scd` clip; the extension selects the reader. If `name` is omitted, the filename becomes the clip name.

11. `Bmc.saveClipScd(name, path)`

    Explicitly saves or resaves an in-memory clip in the complete, human-readable timestamp/message format. When `path` is omitted, the file is saved as `name.scd` in the default `BmcClips` directory. Ordinary `Bmc.record` already performs this save automatically when `Bmc.stopRecording` is called; use `Bmc.saveClipScd` when an explicit path is required or an existing in-memory clip must be written again.

    ```
    Bmc.record(\take1);
    // perform the motion
    Bmc.stopRecording;
    // take1.scd now exists in BmcClipLibrary.defaultDirectory
    ```

12. `Bmc.loadClipScd(path, name)`

    Loads a readable `.scd` clip explicitly. The first stored timestamp is normalized to zero while all frame intervals are preserved. Load only trusted `.scd` files because their message lines are interpreted as SuperCollider code.

    ```
    Bmc.loadClipScd(
        BmcClipLibrary.defaultDirectory +/+ "take1.scd",
        \take1
    );
    ```

13. `Bmc.clipToScd(name)` / `Bmc.convertClipToScd(name)`

    Exports a recorded clip as a human-readable SuperCollider `.scd` file. The file is placed beside the clip's `.bmc` archive and uses the same base name; for example, `take1.bmc` becomes `take1.scd`. The returned value is the full path of the exported file.

    ```
    ~scdPath = Bmc.clipToScd(\take1);
    ~scdPath.postln;
    ```

    If the named clip is not currently loaded, this method automatically looks for its `.bmc` file in `BmcClipLibrary.defaultDirectory` and loads it. Clips loaded or saved at a custom path are exported beside that remembered path.

    Each frame is written as an OscRecorder-style commented clip-relative timestamp followed by the message's sclang representation:

    ```
    //:--[0.125]
    [ '/bunraku/vmc/frame', 1, 'Avatar', 'source', 2 ]
    ```

The entire clip is stored in one `.scd` file; it is not divided into groups of 1,000 messages. Exporting again replaces an existing `.scd` file of the same name. The exported file can be restored with `Bmc.loadClip` or `Bmc.loadClipScd`.

Playback

1. `Bmc.play(name)` / `Bmc.playClip(name)`

   Plays a named clip. With no name, it plays the current clip.

2. `Bmc.pause` and `Bmc.resume`

   Pause and resume the current player task.

3. `Bmc.stopPlayback`

   Stops playback without shutting down the receiver or other Bmc services.

4. `Bmc.seek(seconds)`

   Moves the player's next frame position to the closest frame at or before the requested time.

5. `Bmc.rate(value)`

   Sets playback speed. `1.0` is original timing, `0.5` is half speed, and `2.0` is double speed. The value must be greater than zero.

6. `Bmc.loop(flag)`

   Turns repeated playback on or off.

Combining recordings

1. `Bmc.combineClips(target, source, bones, result, startIndex)`

   Copies selected bones from one clip into another and stores the result as a new `BmcAnimationClip`.

   ```
   Bmc.combineClips(
       \baseTake,
       \armTake,
       \leftArm,
       \combined
   );
   ```

   Built-in body groups include `\leftArm`, `\rightArm`, `\arms`, `\leftLeg`, `\rightLeg`, `\legs`, `\torso`, `\upperBody`, and `\all`. An array of exact bone names can also be supplied.

2. `Bmc.combine(targetFrame, sourceFrame, bones)`

   Lower-level operation that copies selected bones between two individual Bunraku frames.

3. `Bmc.rseq(targetSequence, sourceSequence, bones, startIndex)`

   Lower-level sequence operation that replaces selected bones over a range of frames.

4. `Bmc.bone(frame, boneName)`

   Returns the seven transform values for one named bone in a frame.

Live composition

1. `Bmc.wire(source, bones, target, sourceAvatar, priority)`

   Creates a persistent live routing rule. For example:

   ```
   ~armWire = Bmc.wire(
       "camera-a",
       \leftArm,
       \composite,
       "performer-a"
   );
   ```

Matching left-arm transforms are copied into the `\composite` avatar. Other bones retain the avatar's current or reference pose.

2. `Bmc.unwire(wire)`

   Removes one wire object.

3. `Bmc.listWires`

   Posts and returns the active wires.

4. `Bmc.clearWires`

   Removes every live wire from every Bmc avatar.

5. Bmc class overview

Most users can work entirely through `Bmc`. These supporting classes explain how responsibilities are divided and provide extension points for advanced work

`Bmc`

The public live-coding facade. It owns the current working environment and translates short user commands into operations on the dispatcher, recorder, clip library, player, avatars, and wires. It also retains lower-level frame and sequence combination methods.

`BmcDispatcher`

Receives `/bunraku/vmc/frame` OSC messages, validates them, counts rejected or discontinuous frames, and routes valid frames to registered destinations and avatars.

`BmcAvatar`

Represents one controllable output character. It holds the current pose and a neutral reference pose, completes missing data, applies live wires, and sends completed frames to a function or network destination.

`BmcFrame`

A typed representation of one Bunraku animation frame. It contains protocol version, avatar, source, frame number, encoder timestamp, and a `BmcPose`. It converts between class objects and OSC message arrays.

`BmcPose`

The spatial state of a skeleton: a collection mapping standardized bone names to transforms. It can copy selected bones and fill missing bones from another pose.

`BmcBoneTransform`

The position and rotation of one bone, stored as seven values: `x, y, z, qx, qy, qz, qw`.

`BmcBoneSets`

Named body regions used by clip combination and live wiring. It supplies groups such as `leftArm`, `rightArm`, `legs`, `torso`, and `upperBody`.

`BmcClip`

The base class for a timed sequence of frames. It provides frame access, relative times, duration, copying, and archive reading or writing.

`BmcMocapClip`

A `BmcClip` produced by recording motion-capture input. It can retain capture metadata such as filters, capture point, performer, and source.

`BmcAnimationClip`

A `BmcClip` produced through editing, body-part combination, or future algorithmic generation rather than direct capture.

`BmcClipRecorder`

Collects validated frames, filters them by avatar and source, preserves relative arrival timing, and returns a `BmcMocapClip` when stopped.

`BmcClipPlayer`

Schedules clip frames according to their recorded timing. It handles play, pause, resume, stop, seek, loop, and rate, and sends frames to an avatar, function, or `NetAddr`.

`BmcClipLibrary`

Maintains the named in-memory clip collection and current selection. It implements clip listing, the clip-list GUI, rename/remove operations, and archive save/load.

`BmcWire`

A live rule connecting a selected source and body region to a target avatar. It can filter by source and source-avatar name and carries a priority for cases where several wires affect the same target.

`BunrakuParser`

An older compatibility parser for symbol-labelled Bunraku messages and control bus layouts. The fixed Bunraku Frame protocol-v1 path primarily uses `BmcFrame`, `BmcPose`, and `Bmc.bone` instead.

Further testing and troubleshooting

The repository includes four staged communication tests:

1. XR Animator → Godot
2. XR Animator → encoder → OSCGroups → remote BuMoChi
3. SuperCollider playback → decoder → Godot
4. local and remote sources → BuMoChi synthesis → local decoder → local Godot

They are located in:

`PipelineApplications/Communication_Tests/`

Each test includes its port map, startup order, pass criteria, and common failures. When the complete system fails, return to Test 1 and introduce one component at a time.